Manual 4: Deploying Interactive Web Apps & Dashboards on CESNET Kubernetes

Pollination Ecology Group (Handrkov) — Flask, Streamlit, and R Shiny with Automatic HTTPS

2026-11-12

Table of contents

1. How Web Hosting Works on CESNET K8s 1
2. Step-by-Step: Deploying a Web App 2
   - Step 1: Prepare Your Application Files 2
   - Step 2: Upload Your Source Code into a `ConfigMap` 3
   - Step 3: Configure and Apply `web-app-deployment.yaml` 3
3. Updating a Live Web App (Zero Downtime) 3
4. Deploying Streamlit or R Shiny Apps 4
   - Option A: Streamlit App (`app.py`) 4
   - Option B: R Shiny App (`app.R`) 4
5. Security Best Practices for Public Web Apps 4

Beyond batch scripts, our CESNET Kubernetes namespace (`stenc-ns`) can host **live, 24/7 web applications**—such as image review/annotation portals (e.g., RhizoPic Review, Fly Review), data dashboards, or R Shiny / Python Streamlit apps—with **automatic Let's Encrypt HTTPS certificates** under the domain:

`https://<your-app-name>-stenc.dyn.cloud.e-infra.cz`

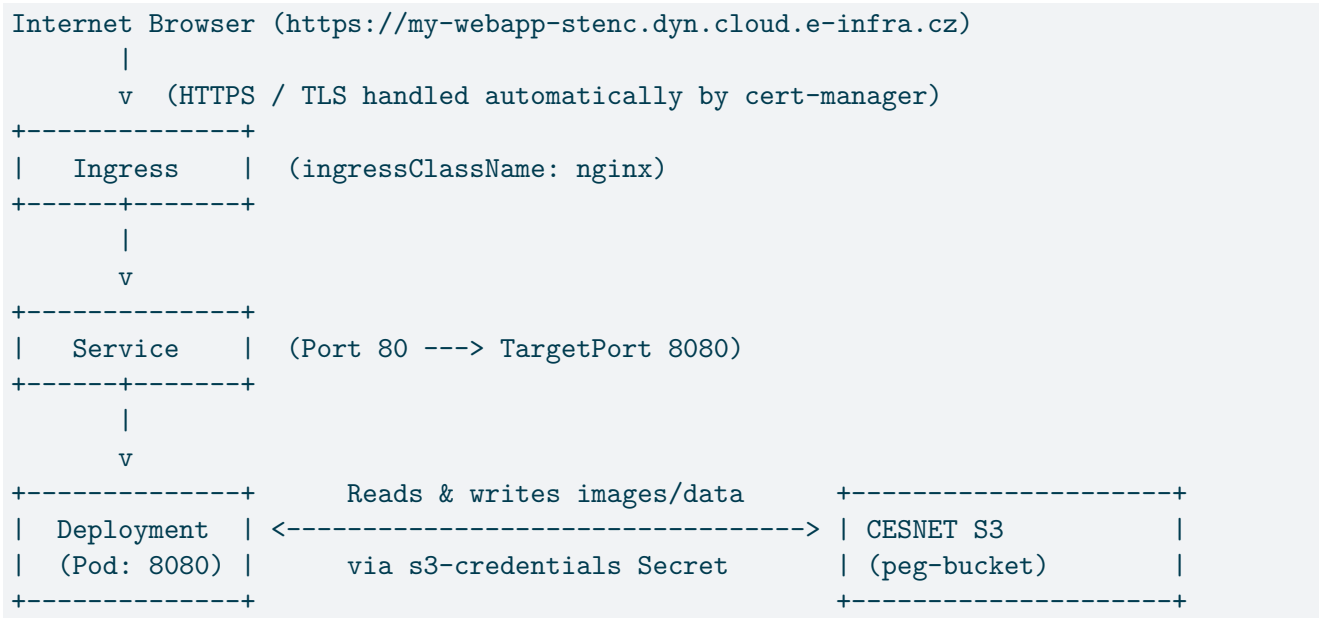
---

1. How Web Hosting Works on CESNET K8s

Whereas a batch script uses a `Job` (which runs once and exits), a web application uses three Kubernetes objects defined together in `templates/k8s/web-app-deployment.yaml`:

1. **`Deployment`**: Runs your web server container (`Flask`/`Waitress`, `FastAPI`/`Uvicorn`, `Streamlit`, or `Shiny`) continuously on port `8080`. If the container ever crashes or a cluster node reboots, Kubernetes automatically restarts it.
2. **`Service`**: Creates a stable internal network address inside `stenc-ns` forwarding port `80` to your container's port `8080`.

3. **Ingress**: Connects the public domain `https://<your-app-name>-stenc.dyn.cloud.e-infra.cz` to your `Service` and automatically provisions a free Let's Encrypt SSL/TLS certificate via `cert-manager`.



---

2. Step-by-Step: Deploying a Web App

Step 1: Prepare Your Application Files

For a lightweight Python web app (e.g., `app.py` and `index.html`), you don't even need a custom Docker image—you can store the source files in a `ConfigMap` and stream heavy images/data directly from `peg-bucket` via `boto3`!

Minimal `app.py` example (listening on port `8080` with a `/healthz` endpoint for Kubernetes readiness checks):

```python
# app.py
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/healthz")
def healthz():
    return jsonify({"status": "ok"}), 200

@app.route("/")
def index():
    return "<h1>Hello from CESNET Kubernetes!</h1>"
```

Step 2: Upload Your Source Code into a `ConfigMap`

```
kubectl create configmap my-webapp-src -n stenc-ns \
  --from-file=app.py=app.py \
  --dry-run=client -o yaml | kubectl apply -f -
```

(If you also have an `index.html` or helper scripts, simply add `--from-file=index.html=index.html` to the command above.)

Step 3: Configure and Apply `web-app-deployment.yaml`

1. Copy `templates/k8s/web-app-deployment.yaml` to your project folder.
2. Replace `my-webapp` with your app's name (e.g., `pollen-viewer`), and set the Ingress host to `<your-app-name>-stenc.dyn.cloud.e-infra.cz`.
3. Apply the manifest:
   ```
   kubectl apply -f web-app-deployment.yaml
   ```
4. Wait for the pod to become ready:
   ```
   kubectl rollout status deployment/my-webapp -n stenc-ns
   ```

Within 30–60 seconds, `cert-manager` will issue the HTTPS certificate and your app will be live at `https://<your-app-name>-stenc.dyn.cloud.e-infra.cz`!

---

3. Updating a Live Web App (Zero Downtime)

Whenever you edit `app.py` or `index.html`, run these two commands to update the `ConfigMap` and perform a rolling restart of the deployment:

```
# 1. Update ConfigMap with the modified files
kubectl create configmap my-webapp-src -n stenc-ns \
  --from-file=app.py=app.py \
  --dry-run=client -o yaml | kubectl apply -f -

# 2. Trigger a rolling restart so the pod loads the new ConfigMap
kubectl rollout restart deployment/my-webapp -n stenc-ns
kubectl rollout status deployment/my-webapp -n stenc-ns
```

---

4. Deploying Streamlit or R Shiny Apps

You can use the exact same `templates/k8s/web-app-deployment.yaml` structure for **Streamlit** or **R Shiny** by changing only the `image:` and `command:` fields:

Option A: Streamlit App (`app.py`)

```
image: python:3.12-slim
command:
- sh
- -c
- |
  pip install --user --quiet --no-warn-script-location streamlit boto3 pandas plotly
  /tmp/.local/bin/streamlit run /app/app.py \
    --server.port=8080 \
    --server.address=0.0.0.0 \
    --server.headless=true
```

Option B: R Shiny App (`app.R`)

```
image: rocker/shiny:4.4.1
command:
- sh
- -c
- |
  mkdir -p /tmp/R_libs
  export HOME=/tmp R_LIBS_USER=/tmp/R_libs
  Rscript -e '
    if (!requireNamespace("aws.s3", quietly=TRUE)) {
      install.packages("aws.s3", repos="https://cloud.r-project.org", lib=Sys.getenv("R_
    }
    shiny::runApp("/app/app.R", host="0.0.0.0", port=8080)
  '
```

---

5. Security Best Practices for Public Web Apps

Because `*.dyn.cloud.e-infra.cz` URLs are reachable from the internet:

1. **Always require login credentials** if your web app can modify data, save annotations back to S3, or display unpublished datasets. Store user passwords/tokens in a Kubernetes `Secret` (never hardcoded in `ConfigMap` or Git), just like our `rhizo-review-users` secret:

```
kubectl create secret generic my-webapp-users -n stenc-ns \
  --from-file=users.json=users.json \
  --dry-run=client -o yaml | kubectl apply -f -
```

2. **Never expose your S3 keys to the browser frontend:** Keep all `boto3` / `aws.s3` calls inside the backend Python/R server, or proxy S3 image streams through your backend endpoints.